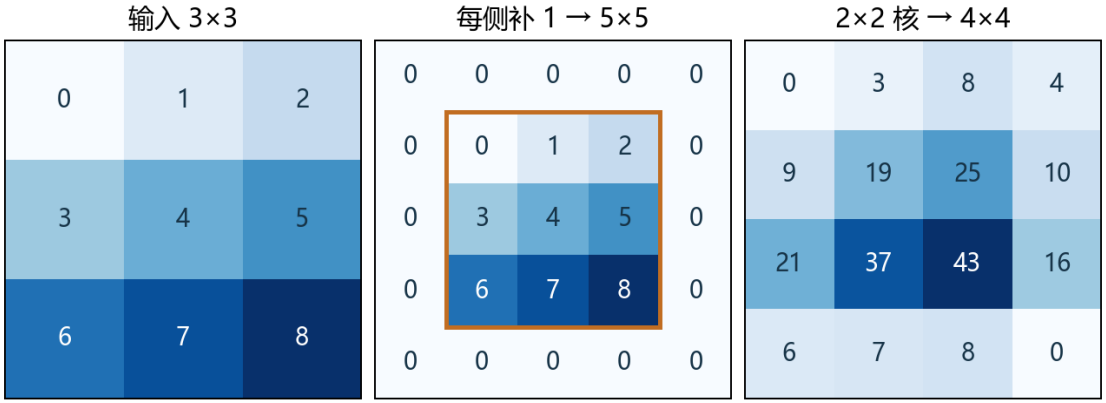
6.3 填充和步幅

教材 PDF 246-249 · 正文 228-231

01 窗口规则确定后，还要决定在哪里计算

6.2 已经用同一个核对每个完整窗口乘加，并从目标响应中学回差分规则。现在窗口内部的计算继续沿用。新增的两个选择是边界外补多少值，以及相邻窗口起点相隔多少格，它们分别由填充 padding 和步幅 stride 控制。



橙框内是原始 3×3 输入，外圈补零后变成 5×5，同一个 2×2 核产生 4×4 响应。

原输入只能放下 2×2 个完整窗口。上下左右各补 1 后，可以放下 4×4 个窗口。首格窗口全部为 0，响应也为 0。补零为窗口提供了边界外的输入值，原始像素由此能够进入更多边界窗口。

填充要解决什么问题

步幅为 1 的 5×5 卷积，每层让高和宽各减少 4。连续十层会各减少 40。若希望较深网络继续保留空间尺寸，可以在每层增加相应边界填充。

每侧补 1、2×2 核的本图中，每个原始像素进入四个窗口。边界响应同时包含图外取零的假设，其值取决于输入边缘与核权重。

两种量怎样对应

教材公式的 P_h、P_w 表示上下、左右的总填充量。PyTorch 的二元 padding=(p_h,p_w) 表示上下各补 p_h、左右各补 p_w。对称填充时 P_h=2*p_h，P_w=2*p_w。

后面的公式统一写总填充量，操作时再转换成所用接口的参数。

6.3 填充和步幅

02 从完整窗口条件推导输出尺寸

设输入高宽为 H、W，核高宽为 k_h、k_w，上下总填充 P_h，左右总填充 P_w。窗口每次向下走 s_h、向右走 s_w。高度上的第 i 个窗口起点为 i*s_h，要放下完整窗口，必须满足下式。

结果 / 结构

```
i*s_h + k_h <= H + P_h

i <= (H + P_h - k_h) / s_h

H_out = (H + P_h - k_h) // s_h + 1
W_out = (W + P_w - k_w) // s_w + 1
```

合法起点从 i=0 开始，所以最大整数索引加 1 才是位置数。Python 的 // 对这里的非负整数做向下整除。最后留下不足一个完整窗口的部分，不能再增加一个输出。

以上假设核非空、步幅为正整数、填充非负，填充后的输入能容纳核，膨胀率为 1。每个方向可以独立计算，再把高宽组合起来。

先列起点，再代公式

8×8 输入、3×3 核、无填充、步幅 2。每方向合法起点为 0、2、4。下一个起点 6 需要读取索引 8，已超出 0 到 7 的输入范围，所以输出为 3×3。

教材题 1 的完整代入

最后一例输入 8×8，核 3×5，PyTorch 单侧填充 (0,1)，步幅 (3,4)。先转换成总填充 (0,2)。

结果 / 结构

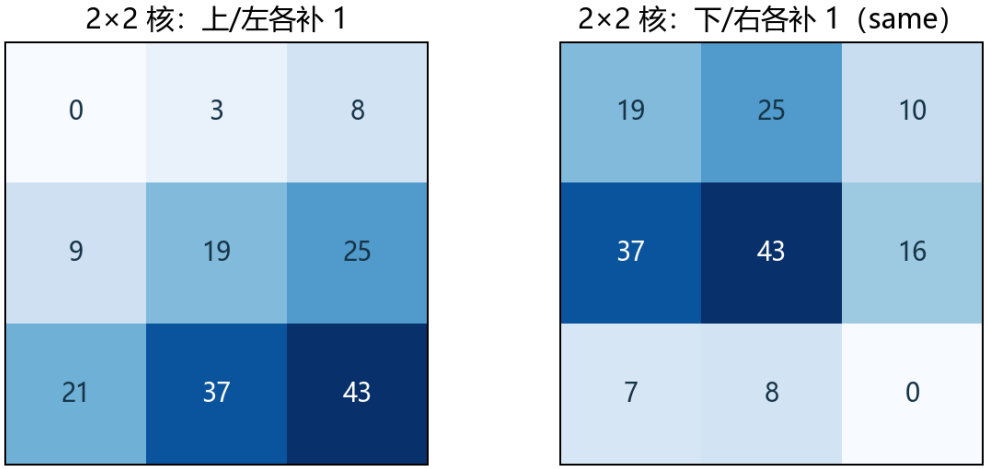
```
H_out = (8+0-3)//3 + 1 = 2
W_out = (8+2-5)//4 + 1 = 2
```

高度起点为 0、3，宽度起点为 0、4，均按填充后的网格计数。列起点与代公式都得到两行两列，这也核对了填充量的记法。

6.3 填充和步幅

03 保持尺寸还要选择对齐方式

步幅为 1 时，令输出尺寸等于输入尺寸，可以从公式解出每方向总填充量等于核大小减 1。3×3 核每方向总共补 2，上下、左右各补 1 即可。5×3 核则上下各补 2、左右各补 1。



2×2 核每方向总填充为 1。上/左多补与下/右多补都保持 3×3，具体响应不同。

奇数核有一个中心，采用对称填充时，输出 (i,j) 的窗口可围绕输入 (i,j) 对齐。偶数核没有唯一中心，每方向所需的奇数总填充量要分到两侧，因而必须选定多补的一侧。

两个接口的参数顺序

F.pad 的四个量按左、右、上、下排列。Conv2d 的二元 padding 按高、宽排列，每个方向两侧对称补齐。

教材给出偶数核上、左多补的可选方案。本机 PyTorch 2.4.1 的 padding="same" 采用下、右多补。形状相同的两张响应，其位置对应关系因此也要核对。

步幅变化后的尺寸

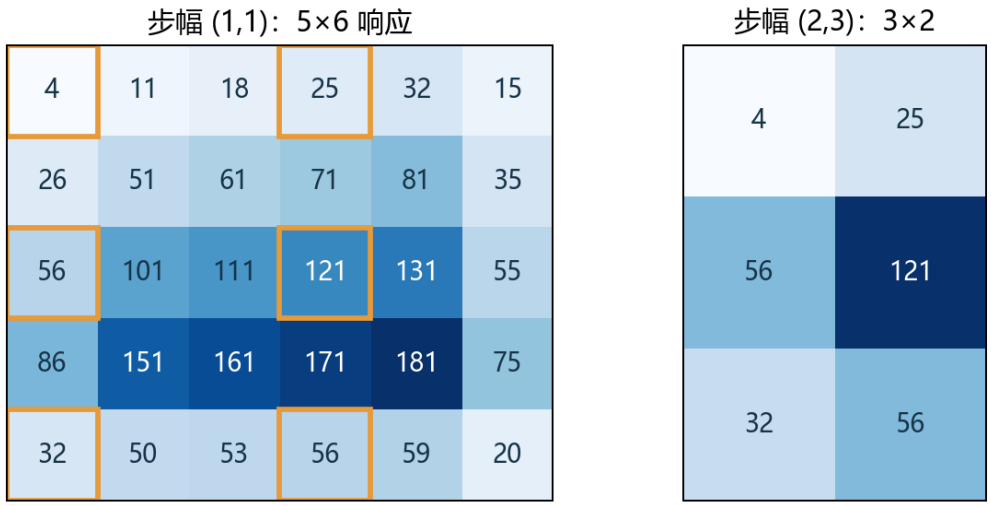
当总填充仍为核大小减 1 时，输出高为 ceil(H/s_h)。7×7 输入、3×3 核、每侧补 1、步幅 2，输出高宽均为 4。输入能被步幅整除时才恰好是 H/s_h。

本机版本 same 只支持步幅 1。需要跨步时，用数值填充或 F.pad 明确安排窗口，再代入尺寸公式。

6.3 填充和步幅

04 步幅把输出索引映射为窗口起点

输出 (i,j) 对应填充后网格的起点 (i*s_h,j*s_w)。若换回原始图像坐标，还需分别减去上、左填充量。窗口内仍执行与 6.2 一样的乘加，变化在于下一次从哪里取窗口。



密集响应中橙框对应步幅 (2,3) 实际计算的位置，右图将这些值按输出索引重新排列。

输入为 4×5、数值按行从 1 到 20，核为 [[1,2],[3,4]]，每侧补 1。步幅 1 得到 5×6 响应。改用 (2,3)，得到 [[4,25],[56,121],[32,56]]，形状为 3×2。

手算中间一个输出

输出 (1,1) 的窗口起点是填充网格 (2,3)，对应原图 (1,2)。窗口 [[8,9],[13,14]] 与核乘加，得到 8+18+39+56=121。

与隔点取样怎样相连

输入、核和填充固定时，步幅 (2,3) 恰好等于密集响应 dense[::2,::3]。这里的 ::2 和 ::3 分别每隔两行、三列取一个值。实际跨步卷积可以直接计算这些位置。

这也改变了 6.1 的位置关系。步幅为 2 时，输入移动一个像素可能改变被抽中的响应位置。比较输出平移需要同时说明步幅、位移与边界。

尺寸下降可以节省后续空间计算，但会丢掉密集响应中的一些位置信息。后面的音频例子进一步展示抽样可能混淆不同信号。

6.3 填充和步幅

教材 PDF 246-249 · 正文 228-231

05 将尺寸、梯度与计算量一起解释

填充和步幅决定哪些窗口参与计算，核权重仍由所有输出位置共享。某个权重的梯度继续汇总选中窗口的贡献。原始输入像素若进入多个窗口，就接收多条路径的梯度，完全未被选中窗口读取时，它在这个局部算子中的梯度为零。

F.pad 补入的固定零值没有新增可训练参数。它们可以参与前向乘加，但优化器更新的是层中登记的核和偏置。改变填充或步幅，也不会自动改变核包含多少个权重。

单输入、单输出通道的 3×3 核有 9 个权重。8×8 输入每侧补 1，步幅 1 有 64 个输出，需要约 576 次乘加。步幅 2 有 16 个输出，需要约 144 次乘加。输出存储和后续层的空间计算量也随之减少。

这里统计的是理论乘加次数。实际时间还取决于实现、输入规模与设备。本节数值实验会另核对前向、输入梯度和核梯度，检验填充及跨步索引后的计算链。

六组配置的连续核对

下表均为 8×8 输入，填充按 PyTorch 单侧高宽排列。

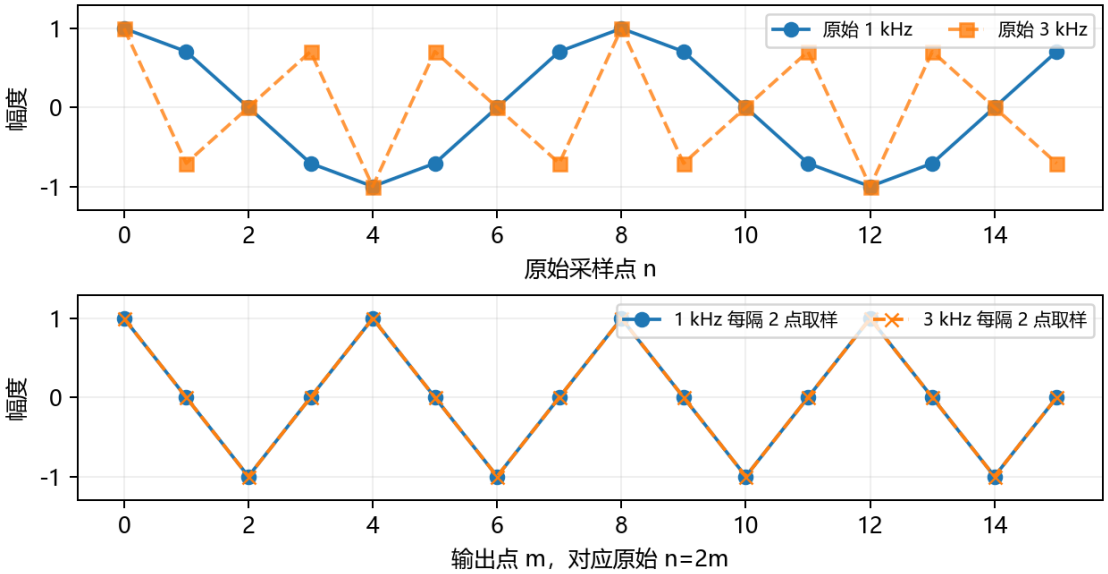
核	填充	步幅	输出
3×3	0,0	1,1	6×6
3×3	1,1	1,1	8×8
5×3	2,1	1,1	8×8
3×3	1,1	2,2	4×4
3×5	0,1	3,4	2×2
3×3	0,0	2,2	3×3

教材题 2 可沿这张表改参数，先算总填充量、列窗口起点，再运行核对。题 4 的计算优势也可以按输出位置数逐项计数。

6.3 填充和步幅

06 将跨步计算理解为采样

音频用离散采样点表示随时间变化的信号。8 kHz 采样时，相邻输入点间隔 0.125 ms。步幅 2 使相邻输出中心相隔两个输入采样间隔，即 0.25 ms，对应 4 kHz 的输出时间网格。



8 kHz 原始网格上的 1 kHz 与 3 kHz 余弦不同，隔两个点取一次后，两条输出重合。

本例低频为 cos(2*pi*0.125*n)，高频为 cos(2*pi*0.375*n)。代入 n=2*m 后，两者在整数 m 上相等。不同的原始信号变成同一输出，这种现象称为混叠。

教材题 3 怎样解释

相同时间范围内，输出点数约减半。如果按原时长播放，要使用相应输出采样率。卷积还包含滤波，边界与核对齐会决定实际覆盖哪些采样点。

降低采样率前通常需要限制高频内容。输出采样率 4 kHz 对应的奈奎斯特频率是 2 kHz，原来的 3 kHz 信号会混叠到较低频率。

滤波与跨步怎样衔接

三点平滑核 [.25,.5,.25] 能减弱快速交替的变化。本例先用它滤波并跨步 2，高频/低频输出 RMS 比约为 0.1716。这个短核抑制高频的能力有限，实际抗混叠设计需要按信号与滤波器要求选择。

卷积核可以学习，但是否学成合适低通，要观察实际权重与响应。到此，边界、尺寸、位置、梯度和采样已经形成同一条知识链。

6.3 填充和步幅

07 操作一 • 跑通完整导读并定位结果

用 VS Code 打开项目根目录，选择已有 openmmlab 解释器，再打开 PowerShell 终端。运行中文导读模式。

结果 / 结构

```
$pythonPath = 'D:\Anaconda\envs\openmmlab\python.exe'
& $pythonPath lessons/ch06/03_padding_and_stride.py --guided
```

控制台按补零与对齐、六组尺寸、窗口起点、梯度及音频四个步骤打印结果。打开教师文件依次读 output_shape、corr2d、padding_demo、stride_demo、shape_demo、gradient_demo、audio_demo，再看 main 怎样把它们组合。

需要覆盖重建实测 JSON 和四张图，在命令末尾加 --record。图保存在 reports/ch06/figures/6_3_*.png，数据保存在 reports/ch06/6_3_metrics.json。

应看到的主要读数

观察项	基准结果
六组输出形状	与第 5 页表格一致
跨步响应	[[4,25],[56,121],[32,56]]
前向和梯度最大差	不超过 2.22e-16
隔点余弦最大差	约 2.18e-14
理论乘加次数	576、144

偶数核的 same 可能显示补零副本的框架提示。导读仍会继续核对结果。代码也会主动捕获 same 配步幅 2 的构造错误，以对应当前版本的支持范围。

接下来在 .cache 新建 6_3_follow_along.py，逐页把完整代码接在同一个文件中，每步保存后运行。这样每次都会从文件开头重建输入。

6.3 填充和步幅

08 操作二・亲自核对补零与对齐

把下面完整代码放入跟做文件，按 F5 选择“运行当前 Python 文件”。

PYTHON

```
import torch
from torch.nn import functional as F

X = torch.arange(9., dtype=torch.float64).reshape(3, 3)
K = torch.arange(4., dtype=X.dtype).reshape(2, 2)
x4, k4 = X[None, None], K[None, None]
base = F.conv2d(x4, k4)[0, 0]
padded = F.conv2d(F.pad(x4, (1, 1, 1, 1)), k4)[0, 0]
before = F.conv2d(F.pad(x4, (1, 0, 1, 0)), k4)[0, 0]
after = F.conv2d(F.pad(x4, (0, 1, 0, 1)), k4)[0, 0]
print("无填充", tuple(base.shape), base.tolist())
print("每侧补 1", tuple(padded.shape), padded.tolist())
print("上/左多补", before.tolist())
print("下/右多补", after.tolist())
```

先观察从 2×2 到 4×4 的输出位置增长，再比较两张 3×3 响应。输入与核始终相同，所以数值变化可以对应到填充与对齐。

每行代码读什么

arange(9) 产生 0 到 8，reshape 排成 3×3。dtype 让两个张量都用 float64。None 为单张图和单个核增加批量与通道轴，形成四维输入。

F.pad 按指定四侧补零，F.conv2d 随后在这个新网格中计算。返回结果的 [0,0] 取回单张、单输出通道的二维响应。

无填充结果为 [[19,25],[37,43]]。每侧补 1 的首行是 [0,3,8,4]，形状为 (4,4)。上/左多补的首格为 0，下/右多补的首格为 19。

运行前先画出这两个首格各自对应的窗口。只记下形状，还不足以确定输出元素与原图位置的对应关系。

6.3 填充和步幅

09 操作三 • 列起点后复现步幅

把下段接在跟做文件后面。这里重新设置输入和核，对应第 4 页的示例。

PYTHON

```
X = torch.arange(1., 21., dtype=torch.float64).reshape(4, 5)
K = torch.tensor([[1., 2.], [3., 4.]], dtype=X.dtype)
padding, stride = (1, 1, 1, 1), (2, 3)
x4 = F.pad(X[None, None], padding)
k4 = K[None, None]
dense = F.conv2d(x4, k4)[0, 0]
strided = F.conv2d(x4, k4, stride=stride)[0, 0]
selected = dense[::2, ::3]
print("密集与跨步形状", tuple(dense.shape), tuple(strided.shape))
print("跨步结果", strided.tolist())
print("与密集选点最大差", (strided-selected).abs().max().item())
```

应得到 (5,6)、(3,2)，结果与第 7 页表格相同，最大差为 0。将输出行索引 0、1、2 乘以 2，得到填充网格起点行 0、2、4。列索引 0、1 乘以 3，得到起点列 0、3。

先改变步幅，再改变填充

第一次只将 stride 改成 (1,1)，预测输出变回 5×6。selected 仍固定用 (2,3) 取样，比较行应同时改成 dense 本身，使比较对象与新步幅一致。

第二次恢复 stride=(2,3)，将 padding 改成 (0,0,0,0)，预测输出为 2×2。此时 dense 和 selected 会跟着新的输入边界重建，比较误差仍应为 0。

每次记录输入高宽、核高宽、四侧填充、窗口起点和输出形状。这样能从数据解释输出改变的原因。

完成后恢复本页的参数，在下一页用手写窗口函数复算同一组结果。

6.3 填充和步幅

10 操作四 · 用手写窗口核对内置结果

PYTHON

```
def corr2d(X, K, padding, stride):
    padded = F.pad(X, padding)
    kh, kw = K.shape
    sh, sw = stride
    rows = (padded.shape[0] - kh) // sh + 1
    cols = (padded.shape[1] - kw) // sw + 1
    outputs = []
    for i in range(rows):
        line = []
        for j in range(cols):
            row, col = i * sh, j * sw
            patch = padded[row:row+kh, col:col+kw]
            line.append((patch * K).sum())
        outputs.append(torch.stack(line))
    return torch.stack(outputs)

hand = corr2d(X, K, padding, stride)
torch.testing.assert_close(hand, strided)
print("手写与内置最大差", (hand-strided).abs().max().item())
```

本页代码接在上一页后面，保存并运行同一个文件。手写与内置最大差应为 0。原来窗口起点是 i、j，现在改成 i*sh、j*sw，窗口乘核求和的规则继续沿用 6.2。

顺着索引检查实现

先由 padded.shape 计算完整窗口数，再把输出索引乘以步幅。切片始终取 kh 行、kw 列，stack 将各格组织成响应图。

在教师文件 corr2d 的 patch 行设置断点，观察 i=1、j=1 时 row=2、col=3，窗口 [[8,9],[13,14]]。再看这个窗口如何得到 121。

梯度怎样复现

完整导读的 gradient_demo 会复制相同输入与核，用同一份均方输出损失分别反传。输入梯度差约 1.11e-16，核梯度差约 2.22e-16。

在 hand.square().mean().backward() 之后暂停，查看 X.grad 和 K.grad。F.pad 与跨步切片都在同一计算图里，原始输入和核都能接收到对应梯度。

6.3 填充和步幅

11 操作五 · 用音频采样验证信息变化

在跟做文件末尾继续追加下段代码，再运行整个文件。

PYTHON

```
n = torch.arange(64, dtype=torch.float64)
low = torch.cos(2 * torch.pi * .125 * n)
high = torch.cos(2 * torch.pi * .375 * n)
print("原序列前 8 点", low[:8].tolist(), high[:8].tolist())
print("隔点后最大差", (low[::2]-high[::2]).abs().max().item())
filter3 = torch.tensor([[[.25, .5, .25]]], dtype=n.dtype)
filtered_low = F.conv1d(low[None, None], filter3, stride=2)
filtered_high = F.conv1d(high[None, None], filter3, stride=2)
ratio = (filtered_high.square().mean() / filtered_low.square().mean()).sqrt()
print("平滑后高频/低频 RMS 比", ratio.item())
```

原序列明显不同，隔点后差异约为 2.18e-14。平滑后 RMS 比约为 0.1716。square().mean().sqrt() 是均方根幅度，比较比值能观察这个核对两种频率的不同响应。

将结果接回图像卷积

图像跨步同样会减少密集响应上的采样位置。边缘、细纹理等快速空间变化可能受到抽样影响。核怎样处理局部模式，步幅怎样选择输出位置，需要放在一起解释。

将三节原理重新串起来

6.1 用局部性和共享限制函数结构。

6.2 学习局部窗口内部的权重。

6.3 决定窗口边界与起点网格，控制响应图尺寸和采样密度。

能够从某个输出索引反推出原图窗口，再写出窗口内的乘加，就能把结构、计算和空间位置对应起来。最后用综合练习检验这一整条过程。

6.3 填充和步幅

12 综合练习与集中反馈

打开 exercises/ch06/03_padding_and_stride.py。输入为 5×6、数值按行从 1 到 30，核为 [[1,0,-1],[1,0,-1]]。四侧填充为 (1,2,0,1)，步幅为 (2,3)。这一题把对称填充换成四侧不同，沿用相同窗口原理。

先在纸上写出补零后的高宽，再列出全部合法窗口起点。TODO 1-3 负责补零与行列数，TODO 4-7 负责从输出索引找到窗口并乘加，TODO 8 用内置运算核对。

结果 / 结构

```
$pythonPath = 'D:\Anaconda\envs\openmmlab\python.exe'
& $pythonPath exercises/ch06/03_padding_and_stride.py
```

如果遇到尺寸差异，先检查四侧填充顺序，再检查整除后是否加 1，最后比较内置输入是否已显式补零。遇到数值差异，先手算首格与最后一格，确认窗口起点和核的对应顺序。

集中反馈

项目	你的预测或实测
填充后的高宽	_____
输出形状和第一行	_____
手写与内置误差	_____
最后一行读取的补零	_____

再说明位置数为何包含第零个窗口、偶数核怎样保持尺寸、音频跨步为何需要考虑混叠。教材题 1、2、3、4 分别贯穿第 2、5、6、5 页。

下一节 6.4 多输入多输出通道从教材 PDF 第 250 页、正文第 232 页开始。

页码依据用户提供的 Release 2.0.0 中文教材，作者为 Aston Zhang、Zachary C. Lipton、Mu Li、Alexander J. Smola。基准与复核保存在 reports/ch06/6_3_metrics.json 和 6_3_verification.md。教材 6.3 PyTorch 2.4.1